

```

# petroleum_formulas.py

# This program uses Object-Oriented Programming (OOP)
# to model 6 important formulas used in Petroleum Engineering.

# Each formula is represented as a class with a `calculate()` method.
# All formula classes inherit from a base class for structure and reusability.

# -----
# Base Formula Class
# -----
class PetroleumFormula:
    def calculate(self):
        # Every child class must implement its own calculate method    raise
        NotImplementedError("This formula needs a calculate() method.")

# -----
# 1. Ideal Gas Law:  $P = nRT / V$ 
# -----
class IdealGasLaw(PetroleumFormula):
    def __init__(self, moles, gas_constant, temperature, volume):
        self.n = moles
        self.R = gas_constant
        self.T = temperature
        self.V = volume

    def calculate(self):
        try:
            pressure = (self.n * self.R * self.T) / self.V
            return round(pressure, 2)
        except ZeroDivisionError:
            return "Error: Volume cannot be zero."

# -----
# 2. Darcy's Law:  $Q = (kA\Delta P) / (\mu L)$ 
# -----
class DarcysLaw(PetroleumFormula):
    def __init__(self, permeability, area, pressure_drop, viscosity, length):
        self.k = permeability
        self.A = area
        self.deltaP =

```

```

pressure_drop    self.mu =
viscosity        self.L = length

    def calculate(self):
try:
    flow_rate = (self.k * self.A * self.deltaP) / (self.mu * self.L)
return round(flow_rate, 4)    except ZeroDivisionError:
return "Error: Viscosity and length cannot be zero."

```

```

# -----

```

```

# 3. Hydrostacc Pressure:  $P = \rho gh$ 

```

```

# -----

```

```

class HydrostaccPressure(PetroleumFormula):
def __init__(self, fluid_density, gravity, height):
self.rho = fluid_density    self.g = gravity
self.h = height

```

```

    def calculate(self):    pressure =
self.rho * self.g * self.h
    return round(pressure, 2)

```

```

# -----

```

```

# 4. Oil Formaon Volume Factor:  $Bo = V_{res} / V_{std}$ 

```

```

# -----

```

```

class FormaonVolumeFactor(PetroleumFormula):    def
__init__(self, reservoir_volume, standard_volume):
    self.V_res = reservoir_volume
    self.V_std = standard_volume

```

```

    def calculate(self):
try:
    Bo = self.V_res / self.V_std
return round(Bo, 3)    except
ZeroDivisionError:
    return "Error: Standard volume cannot be zero."

```

```

# -----

```

```

# 5. API Gravity:  $API = (141.5 / SG) - 131.5$ 

```

```

# -----

```

```

class APIGravity(PetroleumFormula):
def __init__(self, specific_gravity):

```

```

        self.sg = specific_gravity

    def calculate(self):
    try:
        api = (141.5 / self.sg) - 131.5
    return round(api, 2)    except
    ZeroDivisionError:
        return "Error: Specific gravity cannot be zero."

# -----
# 6. Produccvity Index:  $PI = Q / (P_{res} - P_{wf})$ 
# -----
class ProduccvityIndex(PetroleumFormula):    def __init__(self,
flow_rate, reservoir_pressure, wellbore_pressure):
    self.Q = flow_rate
    self.P_res = reservoir_pressure
    self.P_wf = wellbore_pressure

    def calculate(self):
    try:
        PI = self.Q / (self.P_res - self.P_wf)
    return round(PI, 3)    except
    ZeroDivisionError:
        return "Error: Reservoir pressure and wellbore pressure cannot be equal."

# -----
# Polymorphic funccon to display results
# -----
def print_formula_result(formula_object: PetroleumFormula, formula_name: str):
    print(f"💠 {formula_name} Result: {formula_object.calculate()}")

# -----
# Main Program
# -----
if __name__ == "__main__":
    print("👤 Welcome to the Petroleum Engineering Formula Calculator!\n")

    # Create objects for each formula with sample values
    formulas = [
        ("Ideal Gas Law", IdealGasLaw(moles=1, gas_constant=8.314, temperature=350,
volume=22.4)),

```

```

        ("Darcy's Law", DarcysLaw(permeability=100, area=50, pressure_drop=10, viscosity=1.2,
length=30)),
        ("Hydrostacc Pressure", HydrostaccPressure(fluid_density=1000, gravity=9.81,
height=300)),
        ("Formacon Volume Factor", FormaconVolumeFactor(reservoir_volume=1.25,
standard_volume=1)),
        ("API Gravity", APIGravity(specific_gravity=0.85)),
        ("Produccvity Index", ProduccvityIndex(flow_rate=500, reservoir_pressure=3000,
wellbore_pressure=1000))
    ]

```

```

    # Loop through each formula and print results
    for name, formula in formulas:
        print_formula_result(formula, name)

    print("\ All calculacons completed Thank you.")

```

Exercise 1

```

print("hello")
name=input("what is your name? ") ("what problem do you have?") print("enter a num to
test if even or odd") num=intsa(input("enter num"))
if num%2==0:

    print("number is even")
    print("you are gooded")
    print("thank you have a wonderful day")

else:
    print("number is odd") print("you are a fool go and die") print("have a wonderful day")

```